

NLP Assignment 1 Report

Muhammad Umer Siddiqui
ms08702@st.habib.pk
Dhanani School of Science and
Engineering, Habib University
Karachi, Pakistan

Abdullah Shaikh
as09245@st.habib.pk
Dhanani School of Science and
Engineering, Habib University
Karachi, Pakistan

Arqish Zaria
az09714@st.habib.pk
Dhanani School of Science and
Engineering, Habib University
Karachi, Pakistan

Abstract

This report covers both questions of NLP Assignment 1. Question 1 presents Byte Pair Encoding (BPE) and WordPiece tokenizers implemented from scratch for a small multilingual corpus containing English, Urdu and Chinese, and studies the effect of vocabulary size (250 to 900) on the learned subword vocabulary. BPE began learning longer subword strings at smaller vocabulary sizes, while WordPiece required a larger vocabulary, and a vocabulary size of 700 was selected. Question 2 analyses Jane Austen's *Emma* using NLTK: preprocessing reduced the text from 191,855 tokens to 70,171 tokens and the vocabulary from 7,378 to 6,831 unique words, the most frequent words and nouns are dominated by character names, and POS tagging was performed sentence by sentence on the unfiltered text to preserve capitalization and context.

Keywords

Natural Language Processing, Tokenization, BPE, WordPiece, Multilingual NLP, Text Preprocessing, POS Tagging

1 Question 1: BPE and WordPiece Tokenizers

1.1 Introduction

The question provided a small multilingual corpus containing English, Urdu, and Chinese text. The task was to implement Byte Pair Encoding (BPE) and WordPiece tokenizers from scratch and experiment with their hyperparameters on the given corpus. The assignment specifically required consideration of the small corpus size, the three different writing systems, code-switching, Unicode representation, and the trade-off between vocabulary size and token fragmentation.

Our implementation used vocabulary size as its only explicit hyperparameter. The algorithms were implemented in `q1.py`, and the resulting vocabularies were stored in `bpe_vocab.txt` and `wordpiece_vocab.txt`. We experimented with vocabulary sizes of 250, 300, 350, 450, 500, 600, 700, 800, and 900 to examine how the learned subword units changed as the vocabulary grew.

1.2 Preprocessing

The corpus was first normalized using Unicode Normalization Form C (NFC). This provides a canonical representation for characters that can have multiple Unicode representations while preserving their linguistic content. This was particularly important for a multilingual corpus because the tokenizer operates directly on Unicode characters.

The text was then converted to lowercase. This primarily affected the English portion of the corpus, where uppercase and lowercase versions of the same letter would otherwise be treated as different

characters. Lowercasing did not produce a comparable change for the Urdu and Chinese portions of the corpus.

Punctuation was separated from surrounding text rather than removed. This allowed punctuation to remain available as tokens while preventing it from being merged directly with adjacent words. Punctuation was identified using Unicode character categories, making this step independent of any particular language or writing system.

Finally, whitespace was normalized to single spaces. This provided a consistent representation of word boundaries and allowed the implementation to use whitespace-based splitting without being affected by repeated spaces, tabs, or line breaks.

These preprocessing decisions were intended to normalize the corpus without removing linguistic information. In particular, punctuation was retained rather than discarded, and no stemming, lemmatization, stopword removal, transliteration, or language-specific word normalization was performed. Such transformations could alter the frequency distribution of characters and subwords that the two tokenizers use when deciding which pairs to merge.

1.3 Tokenization Methods

A parent class called `Tokenizer` was implemented to provide functionality shared by both algorithms. It reads the corpus, performs the preprocessing steps, and provides functionality for writing the resulting vocabulary to a file. The BPE and WordPiece implementations inherit from this class and implement their respective tokenization and vocabulary-building procedures.

1.3.1 WordPiece. The `WordPieceTokenizer` class contains two main methods: `tokenize` and `vocabulary_builder`.

The `tokenize` method initially represents every word as a sequence of individual characters. The first character of each word is stored normally, while every subsequent character is prefixed with `#`. For example, the word `token` is initially represented conceptually as

[t, ##o, ##k, ##e, ##n].

The `vocabulary_builder` method initializes the vocabulary using these character-level tokens. It then repeatedly calculates the frequency of each token and the frequency of every adjacent token pair.

Unlike BPE, WordPiece does not select a pair solely according to its frequency. For two adjacent tokens x and y , the implementation uses the following score:

$$\text{Score}(x, y) = \frac{\text{Freq}(x, y)}{\text{Freq}(x) \text{Freq}(y)}.$$

The pair with the highest score is selected for merging. The two tokens are combined into a new token, with the ## marker removed from the second token. All occurrences of the selected pair are then replaced by the new token, which is added to the vocabulary.

This process continues until the target vocabulary size is reached or no further adjacent pairs are available.

1.3.2 Byte Pair Encoding. The BPETokenizer class follows a similar iterative procedure but uses a different pair-selection criterion.

The tokenize method splits each word into its individual characters without adding WordPiece-style ## markers. For example, the word token is initially represented as

$$[t, o, k, e, n].$$

The vocabulary is initialized using all unique characters in the corpus. The algorithm then calculates the frequency of every adjacent token pair. The pair with the highest raw frequency is selected:

$$(x^*, y^*) = \arg \max_{(x, y)} \text{Freq}(x, y).$$

The selected pair is concatenated into a new token, and all occurrences of the pair are replaced by the new token. The new token is added to the vocabulary, and the process is repeated until the target vocabulary size is reached.

1.3.3 Comparison of Pair Selection. The primary algorithmic difference between the two implementations is therefore the criterion used to select pairs for merging. BPE selects the pair with the highest raw frequency, whereas WordPiece normalizes the pair frequency using the frequencies of the individual tokens.

This difference can cause the two algorithms to learn different subword vocabularies even when they are given the same corpus and target vocabulary size. BPE tends to favor pairs that occur frequently in the corpus, while the WordPiece score can favor pairs that occur frequently relative to the frequencies of their individual components.

1.4 Experiments and Results

The vocabulary size K was varied to examine how increasing the available vocabulary affected the learned subwords. The tested values were 250, 300, 350, 450, 500, 600, 700, 800, and 900.

Table 1: Observed effect of vocabulary size on learned subwords

K	WordPiece	BPE
250	No English merges; only Chinese tokens learned	41 English, 17 Urdu, 4 Chinese multi-character tokens
300	57 Chinese multi-character tokens; no Urdu or English merges	71 English, 35 Urdu, 6 Chinese multi-character tokens
350	Similar to $K = 300$	Continued improvement
450	78 Chinese, 119 Urdu, 10 English multi-character tokens	140 English, 94 Urdu, 28 Chinese multi-character tokens
500	Little observable change	Little observable change
600	78 Chinese, 242 Urdu, 37 English multi-character tokens	218 English multi-character tokens
700	78 Chinese, 242 Urdu, 137 English multi-character tokens	260 English, 185 Urdu, 67 Chinese multi-character tokens
800	Continued improvement	Identical to $K = 700$; BPE saturates at 772 tokens
900	Continued improvement	Identical to $K = 800$

At $K = 250$, WordPiece had not yet merged any English character pairs. It had, however, already learned some Chinese tokens because WordPiece’s normalized score favours rare pairs, which implies Chinese characters occur less frequently individually than English letters in this corpus. In contrast, BPE had already learned longer strings such as word and tokeniz. This indicates that the two pair-selection criteria produced noticeably different vocabularies even at a relatively small vocabulary size.

Increasing K to 300 produced little observable change in the WordPiece vocabulary. BPE, however, learned additional longer strings, including tokenization and process, as well as suffixes such as ing. At $K = 350$, the observed results were similar to those at $K = 300$.

At $K = 450$, WordPiece began producing substantially longer tokens, with particularly noticeable changes in the Urdu portion of the vocabulary. BPE also continued to produce longer subword units. The increase to $K = 500$ produced comparatively little additional change in either vocabulary.

At $K = 600$, WordPiece began learning a substantial number of English subword strings. At the same vocabulary size, BPE was already learning a substantial number of complete words. Increasing K to 700 continued to improve the learned vocabularies for both algorithms. Further increases to 800 and 900 continued this trend

for WordPiece. BPE, however, reached its maximum of 772 tokens by $K = 800$ because every word in the corpus had been fully merged, making $K = 800$ and $K = 900$ identical for BPE.

1.4.1 Multilingual Observations. The multilingual nature of the corpus affected the vocabularies produced by both algorithms. English words are composed from a relatively small alphabet and contain recurring character sequences, allowing frequent sequences to be merged into longer subwords. This is visible in the English examples learned by BPE at relatively small vocabulary sizes.

The Urdu portion of the corpus showed a more noticeable improvement for WordPiece at larger vocabulary sizes. This ordering follows directly from WordPiece's normalized score: the score favours pairs that are rare individually but frequent together. Chinese characters occur less frequently on their own than English or Urdu letters in this corpus, so Chinese pairs receive high scores and are merged first. Urdu pairs are merged next, and English pairs, whose individual characters are the most frequent, are merged last. This demonstrates that the choice of vocabulary size can affect the representation of different languages differently within a shared multilingual vocabulary.

Chinese differs from English and Urdu because its writing system does not use alphabetic word formation in the same way. Consequently, the character-level initialization provides a particularly direct representation of Chinese text. The learned Chinese vocabulary should therefore be interpreted in terms of character sequences rather than assuming that every learned token corresponds to an English-style word.

1.4.2 BPE and WordPiece Comparison. The experiments demonstrate a difference in how the two algorithms use additional vocabulary capacity. BPE began producing longer strings at smaller vocabulary sizes in our experiments, while WordPiece showed a more noticeable increase in longer subword strings at larger values of K . For example, at $K = 300$, BPE had already learned complete tokens such as *tokenization* and *language*, while WordPiece had not yet merged any English character pairs at all. This is because BPE's raw frequency criterion merges them early.

The difference follows from their pair-selection criteria. BPE directly maximizes pair frequency, so frequently occurring sequences are merged first. WordPiece instead considers pair frequency relative to the frequencies of the individual tokens. As a result, a pair that is frequent in isolation is not necessarily selected by WordPiece at the same stage at which it would be selected by BPE.

1.5 Discussion

The experiments show that vocabulary size has a direct effect on the degree of token fragmentation. A small vocabulary limits the number of merges that can be performed and therefore leaves more of the corpus represented by individual characters or short subwords. Increasing the vocabulary size allows additional merges and consequently produces longer learned units.

The results also show that increasing K does not necessarily produce the same degree of improvement at every step. For example, increasing K from 250 to 300 produced a more noticeable change for BPE than for WordPiece, while larger vocabulary sizes produced more substantial changes in WordPiece.

There is therefore a trade-off in selecting K . A very small vocabulary results in greater token fragmentation, whereas a very large vocabulary can allocate vocabulary entries to increasingly specific strings. Since the assignment corpus is small and multilingual, the selected value should provide useful subword units across the different languages rather than simply maximizing the number of learned strings.

Based on the observed results, $K = 700$ was selected as the final configuration. The corpus contains approximately 300 tokens and 247 distinct words. At $K = 700$, BPE had completed 512 of its 584 possible merges, placing it close to a word-level vocabulary. WordPiece had also begun learning a substantial number of longer subword strings across all three languages. Values above 700 produced diminishing returns, and since BPE saturates at 772, choosing a higher K would only benefit WordPiece while adding unnecessary vocabulary entries.

1.6 Conclusion of Question 1

This part implemented BPE and WordPiece from scratch and examined the effect of vocabulary size on their learned subword vocabularies. The two algorithms differed primarily in their pair-selection criteria: BPE selected the most frequent adjacent pair, while WordPiece used a normalized score based on pair and individual-token frequencies.

Experiments across vocabulary sizes from 250 to 900 showed that BPE began learning longer strings at smaller vocabulary sizes, while WordPiece showed more substantial improvements at larger vocabulary sizes. The experiments also demonstrated that vocabulary size affects different languages differently within a shared multilingual vocabulary.

Based on the observed results, a vocabulary size of 700 was selected for the final configuration because it provided a reasonable balance between vocabulary size and the quality of the learned subword units for the given corpus.

2 Question 2: English Text Preprocessing, Frequency Analysis, and POS Tagging

2.1 Introduction

The question required an analysis of an English text; preprocessing it and comparing statistics before and after, finding the most frequent words, running a part-of-speech (POS) tagger to analyse the nouns and discussing how the preprocessing decisions affected the results. We used Jane Austen's *Emma*, loaded with NLTK's Gutenberg corpus reader (*austen-emma.txt*).

The whole analysis is implemented in *q2_final.py*. Besides printing the results, the script writes three output files: *top10_words.txt* (the ten most frequent words after preprocessing), *top10_nouns.txt* (the ten most frequent nouns) and *unique_nouns.txt* (every distinct noun found, sorted alphabetically). All numbers in this part of the report are taken from the output of this script and from these files.

2.2 Part A: Text Preprocessing

The raw text was first tokenized as is with NLTK's *word_tokenize*, with no modification, to obtain baseline statistics. The baseline total

token count includes every token produced by the tokenizer (punctuation included). The baseline unique-word count only counts alphabetic tokens (tokens for which `isalpha()` is true) and is case sensitive so *The* and *the* count as two different words.

One normalization step was then applied before computing the after preprocessing statistics every occurrence of – (Project Gutenberg’s rendering of an em dash, written with no surrounding white-space) was replaced with – surrounded by spaces. Without this step, punctuation next to an unspaced em dash stays attached to the neighbouring word. For example, in “...brought it.–Nobody...” the period stays fused to the word before the dash and the token becomes “it.” which would later fail the alphabetic filter and get mistagged.

After this normalization, every token was lowercased and passed through two filters: an alphabetic check (`isalpha()`) and a stopword filter using NLTK’s built-in English stopwords list. The alphabetic check removes punctuation, digits and any other non-word tokens. Note that it also removes tokens containing an apostrophe or a period, such as contraction fragments (*n’t*, *’s*) and dotted abbreviations such as *Mr.*, so those tokens do not reach the frequency analysis later on.

Table 2: Token and vocabulary counts before and after preprocessing

	Total tokens	Unique words
Before preprocessing	191,855	7,378
After preprocessing	70,171	6,831

Preprocessing removed 63.42% of all tokens (191,855 to 70,171) but only about 7.41% of the unique words (7,378 to 6,831). The token count falls sharply because punctuation and stopwords are the most repeated tokens in the novel whereas each of them is only a single entry in the vocabulary. The unique-word count falls by a much smaller amount because the words removed are few in number and because lowercasing merges case variants (such as *The* and *the*) into one entry which also reduces the count slightly.

2.3 Part B: Word Frequency Analysis

This part identifies the ten most frequent words remaining after preprocessing, computed with a frequency counter over the filtered token list.

Table 3: Top 10 most frequent words after preprocessing

Rank	Word	Frequency
1	emma	865
2	could	837
3	would	819
4	miss	599
5	must	567
6	harriet	506
7	much	486
8	said	484
9	one	447
10	weston	439

The number of words occurring exactly once (hapax legomena) is 2,682, which is about 39.26% of the 6,831 unique words left after preprocessing.

Two things stand out in this list. First, the character names *emma*, *harriet* and *weston*, together with the address term *miss*, take four of the ten places, which is expected in a novel built around a small set of characters and their social interactions. Second, the modal verbs *could*, *would* and *must* survived stopword removal and occupy three of the top five places. NLTK’s default English stopword list does not include these modal auxiliaries so words that behave more like grammatical connectors than content words still pass through the filter. The generic words *much*, *said* and *one* also appear in the list.

2.4 Part C: Part-of-Speech Tagging

Nouns were defined as any token tagged NN, NNS, NNP or NNPS, i.e. singular and plural common nouns together with singular and plural proper nouns. Proper noun tags are included because a large share of the nouns in this text are character names and excluding them would have made the analysis mostly about generic words rather than the people the story is about.

POS tagging was performed on the em-dash-normalized text *before* lowercasing or stopword removal rather than on the preprocessed token list. The text was split into sentences with `sent_tokenize` and each sentence was tokenized and tagged separately with `pos_tag`. Tagging on the filtered, lowercased token list would remove exactly the two signals the tagger relies on: capitalization (needed to recognize proper nouns and titles) and sentence level context (needed to resolve ambiguous words). The em-dash is also applied before tagging because a fused token such as “it.” can be tagged as a noun even though the underlying word is a pronoun.

In addition to the tag based filter, each noun token was required to match the pattern of one or more alphabetic characters with an optional single trailing period. This excludes Project Gutenberg formatting artifacts such as underscore-delimited italics (“*her*”) and stray bracket tokens (“[”, “]”), while still allowing abbreviations such as *Mr.* and single-letter initials to pass through. All nouns were lowercased after tagging so that capitalized and lowercase occurrences of the same word (for example *Time* and *time*) are counted together.

Table 4: Top 10 most frequent nouns (NN, NNS, NNP, NNPS)

Rank	Noun	Frequency
1	mr	1,153
2	emma	857
3	mrs	699
4	miss	584
5	harriet	494
6	weston	439
7	thing	398
8	knightley	389
9	elton	385
10	woodhouse	311

The total number of noun occurrences is 31,317 and the number of distinct nouns is 3,711.

The titles *mr* and *mrs* appear without their trailing period because of an additional normalization step. The tokenizer sometimes splits the period off an abbreviation into its own token (for example when the abbreviation falls at a detected sentence boundary) so both the bare form *mr* and the dotted form *mr.* can occur in the noun list. For every noun token ending in a period, the code checks whether the same word also occurs in the noun list without a period. If it does, the two are merged into a single count under the bare spelling; if no bare form exists (for example *dr.* or *sept.* in `unique_nouns.txt`), the period is kept. This is why the titles in Table 4 report a single combined frequency (1,153 for *mr* and 699 for *mrs*) rather than a count split across two spellings of the same word.

Apart from these two titles, the list consists of six character names (*emma*, *harriet*, *weston*, *knightley*, *elton*, *woodhouse*), the address term *miss* and one generic common noun, *thing*.

2.4.1 Residual Tagging Errors. The full noun list in `unique_nouns.txt` shows that the tagger is not perfect. Besides genuine nouns, it contains some function words and other non-nouns, for example *a*, *am*, *are*, *was*, *were*, *had*, *her*, *him*, *my* and *which*, as well as Roman numerals from chapter headings (*ii*, *iii*, *iv*, ... *xix*) and single-letter initials. These entries are part of the 3,711 distinct nouns reported above, so that figure should be read as an upper bound on the number of true nouns. None of them appears in the top ten of Table 4.

2.5 Part D: Analysis

Effect of preprocessing on vocabulary and frequency. Preprocessing reduced the token count by about 63.42% but the unique-word count by only about 7.41% (Table 2). This gap is expected: a handful of function words and punctuation marks such as *the*, *to*, *and* and commas account for a large share of the raw tokens but each is only one vocabulary entry so removing them shrinks the token count far more than the number of word types.

Effect of stopword removal on the most frequent words. Stopword removal substantially changed the frequency list: the function words that would dominate an unfiltered count, such as *the*, *to* and *and* do not survive it. However, it is not a complete fix. NLTK's default stopword list omits modal auxiliaries so *could*, *would* and *must* still occupy three of the top five places in Table 3, and other

generic words such as *much*, *said* and *one* also remain. The most frequent word is therefore not necessarily the most meaningful word and the choice of stopword list has a direct effect on the frequency table. Also, because the alphabetic filter removes dotted abbreviations, *mr* and *mrs* are largely absent from the word list in Part B, although they are the most frequent nouns in Part C.

Types of nouns that dominate. Excluding the two honorifics, the list in Table 4 consists of the six character names, the address term *miss* and the generic noun *thing*. This fits a novel that is driven mostly by conversation and social relations among characters rather than by description of objects or settings.

Preprocessing decisions that affected POS tagging. Tagging the text before any lowercasing or stopword removal was a deliberate choice: keeping capitalization let the tagger recognize character names and titles such as *Mr.* and *Mrs.* as proper nouns and tagging sentence by sentence preserved the sentence level context the tagger uses to resolve ambiguous words. Lowercasing the nouns only after tagging let case variants be merged into a single count without affecting the tagging decision.

The clearest example of a preprocessing decision affecting tagging is the em-dash handling. A fused token like *"it."*, produced when a word and its period were followed directly by *–*, could be tagged as a noun because of a Project Gutenberg formatting quirk and not because of a grammatical decision by the tagger; the word *it* on its own is a pronoun. Adding spaces around the em dash before tokenizing removes this source of error. The residual errors listed in Section 2.4.1 show that some tagging mistakes still remain so a frequency list should always be checked for unexpected entries after a tagging pass.

2.6 Conclusion of Question 2

This part analysed *Emma* with NLTK. Preprocessing reduced the text from 191,855 to 70,171 tokens (63.42%) and the vocabulary from 7,378 to 6,831 unique words (7.41%). The most frequent remaining words are character names and modal verbs that NLTK's stopword list does not remove, and 2,682 words (39.26% of the remaining vocabulary) occur only once. POS tagging, performed sentence by sentence on the unfiltered text, found 31,317 noun occurrences across 3,711 distinct nouns, dominated by honorifics and character names. The results show that preprocessing choices, in particular the stopword list, capitalization handling and the treatment of the em dash, have a direct effect on both the frequency statistics and the quality of POS tagging.

3 Overall Conclusion

Both questions show that preprocessing decisions shape what downstream methods can learn. In Question 1, preprocessing was deliberately minimal (NFC normalization, lowercasing, punctuation separation and whitespace normalization) so that the BPE and Word-Piece merge statistics reflect the raw multilingual corpus, and the choice of vocabulary size determined how finely the text was fragmented. In Question 2, heavier preprocessing (stopword removal and an alphabetic filter) was appropriate for frequency analysis, but the POS tagging step had to be run on the less processed text so that capitalization and sentence context were preserved.